UNED

Departamento de Sistemas de Comunicación y Control

Bases de datos documentales: MongoDB

Gestión / Almacenamiento de información no estructurada

Máster Universitario en Ingeniería y Ciencia de Datos

Andrés Duque Fernández

LECTURAS PARA PREPARAR EL TEMA

- Libro *MongoDB: The Definitive Guide, 3rd Edition*:
  - https://learning.oreilly.com/library/view/mongodb-the-definitive/9781491954454/
- Documentación oficial MongoDB:
  - https://docs.mongodb.com/
- Los enlaces de los libros son accesibles para usuarios autenticados en UNED

ÍNDICE

- Introducción
- Modelo de datos
- Operaciones básicas
- Operaciones CRUD en MongoDB
- Agregación
- Índices
- Arquitectura de *clustering*
  - *Sharding*
  - Réplicas
- Otras características
- Conclusiones

INTRODUCCIÓN

- Orígenes de MongoDB:
  - Creado en 2007 por la empresa 10gen
  - Lanzado como OpenSource en 2009 bajo licencia AGPL
- Nombre derivado de "humongous" (enorme, extraordinariamente grande)
- Base de datos orientada a documentos:
  - Datos almacenados en documentos en lugar de en registros
- *Schemaless* (sin esquema):
  - Diferencia fundamental con BBDD relacionales
  - Documentos en una misma colección (concepto similar a una tabla en relacional) pueden tener esquemas distintos

INTRODUCCIÓN

- Aplicaciones de MongoDB:
  - Internet de las cosas (*Internet of Things*, *IoT*)
  - Visualización geoespacial
  - Gestión de contenidos (*Sourceforge*)
  - Aplicaciones móviles (*Expedia*)
  - Videojuegos (FIFA online)
  - Log de eventos (*Facebook*)
  - Otros: *Ebay*, *Orange*, *Barclays*, *Adobe*, *Telefónica*…

MODELO DE DATOS

- Documentos en formato BSON:
  - Representación binaria de JSON (similar, con algunas particularidades):
    - Tipos de datos adicionales
    - Más eficiencia en espacio
    - Diseñado para búsquedas más eficientes
  - Más información y tipos de datos: http://bsonspec.org y https://docs.mongodb.com/manual/reference/bson-types/

MODELO DE DATOS

- Tipos de datos:
  - Numéricos:
    - *Double* (coma flotante, 64 bits)
    - *Decimal* (coma flotante, 128 bits)
    - *Int* (enteros, hasta 32 bits)
    - *Long* (enteros, hasta 64 bits)
  - Texto:
    - *String* (UTF-8)
    - *Regex* (almacenamiento de expresiones regulares)
  - Fecha
    - *Date* (entero de 64 bits, milisegundos desde el 1 de enero de 1970)
    - *Timestamp* (valor único, entero de 64 bits, los primeros 32 son los segundos desde el 1 de enero de 1970 y los siguientes son ordinales incrementales)

MODELO DE DATOS

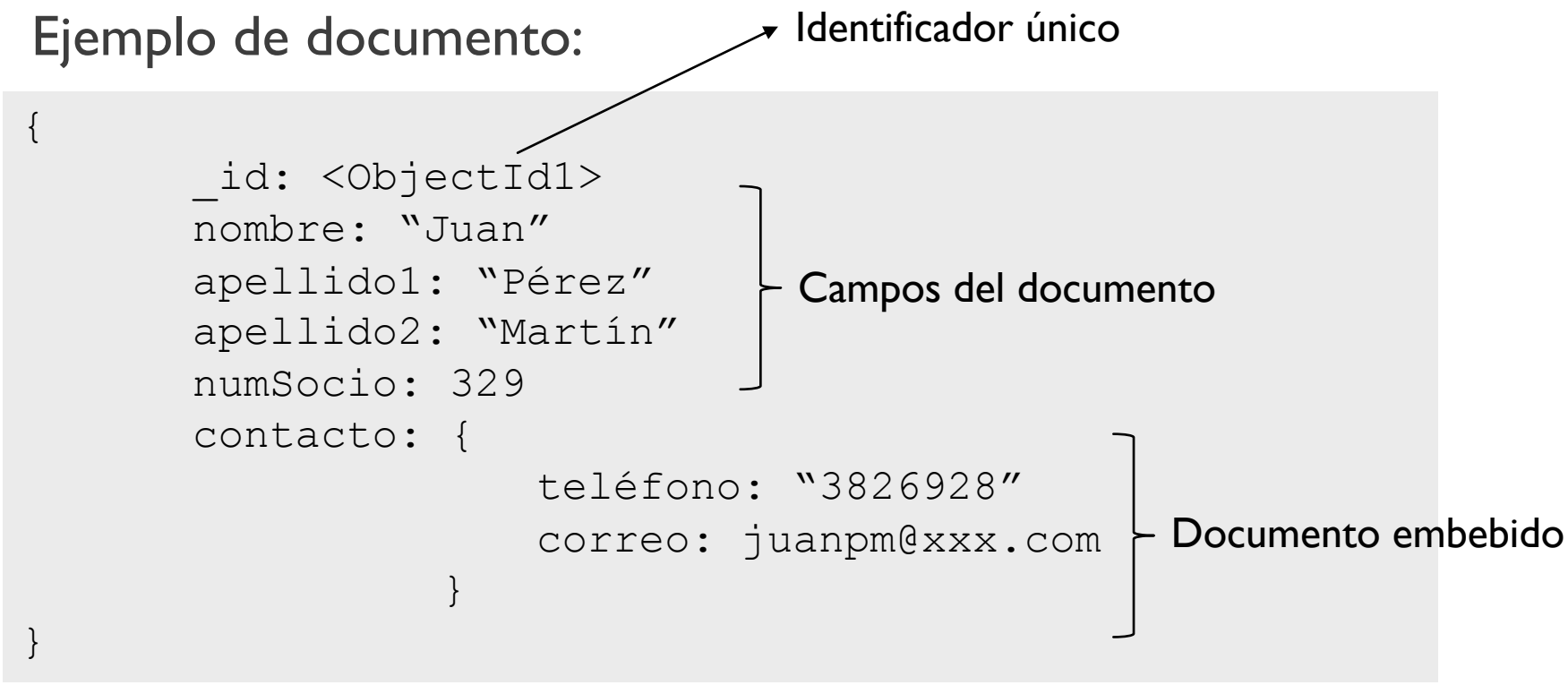
- Tipos de datos especiales:
  - *Array* (conjunto de elementos de cualquier tipo)
  - *ObjectId* (valor único, utilizado para el _id)
  - *Javascript* (código Javascript)
  - *Null* (valor nulo)
  - *Boolean* (valor booleano)

MODELO DE DATOS

- Conceptos básicos:
  - Documento: conjunto de pares campo-valor (similar a filas):
    - Sin esquema predefinido (tantos campos como se desee)
    - Nombre del campo junto a su valor
    - No hay valores nulos (aunque exista el tipo de datos)
    - Pueden contener otros documentos (jerarquía de documentos)
  - Colección: conjunto de documentos (similar a tablas):
    - Sin esquema (cada documento puede tener campos distintos)
    - Identificador único para cada documento dentro de una colección (_id)
  - Campos: tienen un valor asignado (similar a columnas)
    - Se pueden crear índices sobre ellos
    - Campo _id siempre presente (si no se le da valor se genera automáticamente)

MODELO DE DATOS

- Ejemplo de documento:



MODELO DE DATOS

- Factores importantes:
  - No se parte de una metodología claramente establecida
  - Entorno "más rico" que SQL:
    - Tablas embebidas
    - Documentos embebidos con nivel de profundidad arbitrario
    - Claves que pueden faltar
    - Documentos con esquemas distintos en la misma colección
  - Como regla general debemos olvidarnos de:
    - JOINS
    - Foreign Keys
    - Transacciones (aunque sí operaciones atómicas)

MODELO DE DATOS

- Esquema de datos flexible (el desarrollador lo implementa según sus requisitos).
- Tipos de relaciones:
  - *One-to-one*: Uno de los documentos se suele embeber dentro de otro.
  - *One-to-many* embebido: Los documentos se embeben dentro de una estructura de tipo *Array*.
  - *One-to-many* con referencias: Se hace referencia al _id de los documentos embebidos en lugar de embeberlos por completo.
  - *Many-to-many*: Se dejan separadas las dos colecciones.
    - Si la relación es "pocos-a-muchos", se crea un array en la colección "muchos" con referencias a los ids de la colección "pocos".
    - Si la relación es "muchos-a-muchos" se puede crear una tercera colección para representar la relación, como se hace en relacional.

MODELO DE DATOS

- Ejemplos: *One-to-one*
  - Relación socio-dirección:

```
{
        _id: <ObjectId1>
        nombre: "Juan"
        apellido1: "Pérez"
        apellido2: "Martín"
        numSocio: 329
        direccion: {
                        calle: "Avenida de los Conquistadores",
                        numero: 27,
                        ...
                    }
}
```

MODELO DE DATOS

- Ejemplos: *One-to-many* embebido
  - Relación socio-dirección (si se pueden tener varias direcciones):

```
{
        _id: <ObjectId1>
        nombre: "Juan"
        apellido1: "Pérez"
        apellido2: "Martín"
        numSocio: 329
        direcciones: [{
                                calle: "Calle Mayor",
                                numero: 27,
                        ...},
                        {
                                calle: "Calle del Pez",
                                numero: 4,
                        ...},
                        ...]
}
```

MODELO DE DATOS

- Ejemplos: *One-to-many* con referencias
  - Relación libro-editorial

Editorial

```
{
        _id: "edSalamandra",
        ciudad: "Madrid",
        CIF: 2832935,
        ...
}
```

Libro

```
{
        _id: "HdC1",
        nombre: "El perfume",
        ISBN: "348639323598"
        editor: "edSalamandra",
        ...
}
```

MODELO DE DATOS

- Ejemplos: *Many-to-many* (pocos a muchos)
  - Relación trabajador-proyecto

Trabajador

```
{
        _id: "T1",
        Nombre: "Juan",
        Apellido: "Pérez",
        DNI: 12857323F,
        Proyectos: [P1,
                    P2,...],
        ...
}
```

Proyecto

```
{
        _id: "P1",
        nombre: "Proyecto 1",
        ...
}
```

OPERACIONES BÁSICAS

- Creación, uso y borrado de una base de datos:
  - Acceso a la *shell*: comando "mongo"
  - Creación de la base de datos:
    - use <dbName>
  - Utilización de la base de datos:
    - use <dbName>: Acceso a la BBDD a través de la variable "db"
    - Utilización de "db." más el nombre del comando
  - Borrado de la base de datos:
    - use <dbName>
    - db.dropDatabase()

OPERACIONES BÁSICAS

- Colecciones:
  - Creación:
    - Las colecciones se crean al insertar el primer documento
      - db.<collectionName>.insert(…)
    - Si se quieren especificar parámetros se crean con createCollection:
      - db.createCollection(name, options)

OPERACIONES BÁSICAS

- Colecciones:
  - Creación (algunos parámetros):
    - *capped*: booleano para determinar si la colección tiene tamaño máximo (se especifica con la opción *size*)
    - *autoIndexId:* booleano para determinar "_id" manual o automático
    - *max:* número entero para indicar el máximo de documentos en la colección.
    - Consultar todos los parámetros en https://docs.mongodb.com/manual/reference/method/db.createCollection/

OPERACIONES CRUD EN MONGODB

- Operaciones CRUD:
  - Insert: escritura (Create) de datos
  - Find: lectura (Read) de datos (similar a SELECT en SQL)
  - Update: actualización de valores
  - Remove: borrado (Delete) de datos
- Esquema genérico:
  - db.<collectionName>.<operationName>(<conditions>)

OPERACIONES CRUD EN MONGODB

- Creación: *Insert*
  - Si no se inserta valor para _id, se genera automáticamente
  - Ejemplo:

```
db.socios.insert({
                numSocio: 300,
                nombre: "Pablo",
                ...})
```

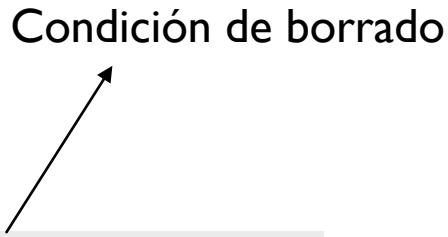
OPERACIONES CRUD EN MONGODB

- Actualización: *Update*
  - $set: utilizado para determinar los nuevos campos
  - Opciones
    - {*multi* = True}: se pueden modificar varios documentos simultáneamente.
    - {*upsert* = True}: si no hay documentos que cumplan la condición se crea uno nuevo.
  - Ejemplo:

```
db.socios.update(
                {numSocio: 300},
                {$set nombre: "Ramón"},
                {<options>}
                )
```

OPERACIONES CRUD EN MONGODB

- Borrado: *Remove* y *Drop*
  - Ejemplo *remove*:




```
db.socios.remove({numSocio:300})
```

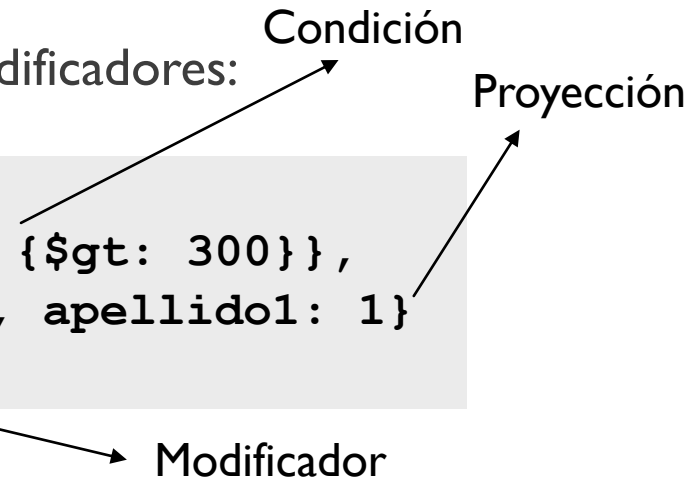
  - Ejemplo *drop* (colecciones):

```
db.socios.drop()
```

OPERACIONES CRUD EN MONGODB

- Lectura: *Find*
  - Dividido en condiciones, proyecciones y modificadores:
    - Ejemplo:

```
db.socios.find(
                {numSocio: {$gt: 300}},
                {nombre: 1, apellido1: 1}
                ).limit(3)
```



    - Documentación en: https://docs.mongodb.com/manual/tutorial/query-documents/

OPERACIONES CRUD EN MONGODB

- Find. Condiciones:
  - Igualdad: $eq
  - Mayor que ($gt) y mayor o igual que ($gte)
  - Menor que ($lt) y menor o igual que ($lte)
  - Diferente a ($ne)
  - Pertenece a ($in) o no pertenece a ($nin), para conjuntos

OPERACIONES CRUD EN MONGODB

- Find. Condiciones:
  - Operadores lógicos:
    - $and, $or, $not y $nor
    - Posibilidad de anidamiento
    - Existencia de campo ($exists)
    - Campo de tipo concreto ($type)
    - Sobre arrays: $all, $elemMatch y $size

OPERACIONES CRUD EN MONGODB

- Find. Proyecciones:
  - Valor 1 para mostrar un campo
  - Valor 0 para no mostrar un campo
  - Si no se declara nada se muestran todos los campos
    - Pero si se declara un valor 1 sólo se muestra dicho campo
    - Campo _id siempre se muestra a menos que se indique valor 0

OPERACIONES CRUD EN MONGODB

- Find. Modificadores:
  - Sort(): Ordenación según un campo dado (valor 1 ascendente y -1 descendente).
  - Limit(): Número máximo de documentos retornados (0 indica sin límite).

AGREGACIÓN

- Count:
  - db.<collection>.count({<field>:<value>}): Número de documentos en los que aparece un campo con un valor concreto.
  - Ejemplo: si tenemos los siguientes documentos en la colección "colec":
    - {puntuacion: 5}, {puntuacion: 3}, {puntuacion: 5}, {puntuacion: 5}, {puntuacion: 2}
    - La siguiente llamada devolverá 3:

```
db.colec.count({puntuacion: 5})
```

AGREGACIÓN

- Distinct:
  - db.<collection>.distinct(<field>): Devuelve un array con los diversos valores que se encuentran para el campo <field>.
  - Ejemplo: para los documentos de la colección anterior:

```
db.colec.distinct("puntuacion")
```

  - Devuelve el array [5,3,2].

AGREGACIÓN

- Aggregate:
  - Permite realizar "queries" anidadas que incluyen operaciones.
  - db.<collection>.aggregate([<pipeline>]): dentro del pipeline se incluyen las diversas operaciones que se pueden realizar:
  - Dentro de dichas operaciones se utilizan expresiones específicas para lograr resultados concretos.

AGREGACIÓN

- Aggregate. Operaciones:
  - $project: utilizado para modificar datos de entrada, añadiendo, eliminando o recalculando datos.
  - $match: filtra la entrada para reducir el número de documentos.
  - $limit: limita el número de documentos de salida.
  - $skip: ignora un número de registros, devolviendo los siguientes.
  - $unwind: convierte un array para devolverlo separado en documentos.
  - $group: agrupa documentos según una condición.
  - $sort: ordena documentos según un campo especificado.
  - $geoNear: documentos ordenados por proximidad (datos geoespaciales).
  - $lookup: realiza un *Left Outer Join* a otra colección de la misma base de datos para procesar la unión.

AGREGACIÓN

- Aggregate. Algunas expresiones:
  - Comparadores como $gt, $lt, $eq, o $ne.
  - Acumuladores como $sum, $avg, $max, $min, $first o $last.
  - Operadores aritméticos como $abs, $exp, $floor, $trunc, $pow o $sqrt.
  - Sobre arrays como $in, $range, $size o $slice.
  - Operadores booleanos como $and, $or o $not.
  - Operadores sobre fechas como $dateToString, $day, $month o $year.
  - Operadores sobre conjuntos como $setUnion o $setIntersection.
  - Operadores sobre strings como $split, $regexFind o $concat.
  - Otros (trigonométricos, de tipo, …)
  - Consultar lista completa en: https://docs.mongodb.com/manual/meta/aggregation-quick-reference/#aggregation-expressions

AGREGACIÓN

- Aggregate. Equivalencias con SQL:

SQL	Aggregate
WHERE	$match
GROUP BY	$group
HAVING	$match
SELECT	$project
ORDER BY	$sort
LIMIT	$limit
JOIN	$lookup

AGREGACIÓN

- Aggregate. Optimización:
  - Si dos comandos se pueden fusionar lo hacen automáticamente.
  - Se debe utilizar $match antes que $sort para ordenar sobre menos documentos.
  - $skip se reordena después de $limit, aunque se coloquen al revés en el pipeline.
  - Dos $limit o dos $skip seguidos se fusionan automáticamente.
  - Es más eficiente anidar $match con operaciones $and que utilizar varios $match en operaciones distintas.

AGREGACIÓN

- MapReduce (menos eficiente que *aggregate*):
  - Función Map / Reduce para realizar consultas y agrupamientos.
    - Fase *Map*: procesado de documentos y generación de salidas para cada documento de entrada.
    - Fase *Reduce*: combina la entrada de la operación *map*.
    - Estructura:

```
db.<collection>.mapReduce(
                function(){emit(key, value);},
                function(key, values){return redFunction},{
                            out: collection
                            query: document
                            sort: document
                            limit: number
                      })
```

Función *map*

Función *reduce*

AGREGACIÓN

- MapReduce. Elementos:
  - Map: función JavaScript que emite pares clave-valor.
  - Reduce: función JavaScript que agrupa (reduce) grupos de documentos con la misma clave.
  - Out: Localización del resultado de la query MapReduce.
  - Query: Campos de selección opcionales.
  - Sort: Criterio de ordenación.
  - Limit: Máximo número (opcional) de documentos devueltos.

AGREGACIÓN

- Más sobre agregación: https://docs.mongodb.com/manual/aggregation/

ÍNDICES

- Utilizados para la optimización de consultas en bases de datos muy grandes.
- Pueden suponer grandes diferencias en tiempos de respuesta.
- Estructuras auxiliares (copias ordenadas de ciertas claves de la colección).
- Sintaxis:

```
db.<collection>.createIndex({<campo>:<orden>,
                             <options>…})
```

ÍNDICES

- Por defecto: existe siempre sobre el campo _id (garantiza unicidad).
- Índice simple:
  - Aplicados a un solo campo de la colección.
  - Sólo se especifica un <campo>.
  - Si <orden> es 1 será ascendente, si es -1 será descendente.

ÍNDICES

- Índice compuesto:
  - Se genera sobre varios campos.
  - Cada campo tiene un orden concreto.
  - Los datos se agrupan en el orden que se definen los campos.
  - Útiles para consultar uno o varios de los campos sin necesidad de incluirlos todos.
- *Multikey index*:
  - Si se declara un índice sobre un campo de tipo *Array*, se crea sobre cada elemento del *Array*.

ÍNDICES

- Opciones en índices:
  - "unique": con valor "true" obliga a tener valores únicos en los campos indexados (si intentamos introducir un nuevo documento con dicho valor repetido, devolverá un error).
  - "sparse": con valor "true" sólo se incluyen en el índice los documentos para los cuáles dicho campo exista (si no se especifica se incluyen todos).
  - "expireAfterSeconds": tiempo de vida (TTL) del índice.
  - "partialFilterExpression": índice creado únicamente sobre documentos que cumplan una condición.

ÍNDICES

- Índices sobre texto:
  - Facilitan y optimizan búsquedas basadas en texto.
  - Se indican poniendo "text" en <orden> (en lugar de 1 o -1).
  - Una colección sólo puede tener **un índice basado en texto:**
    - Pero dicho índice puede ser sobre varios campos.

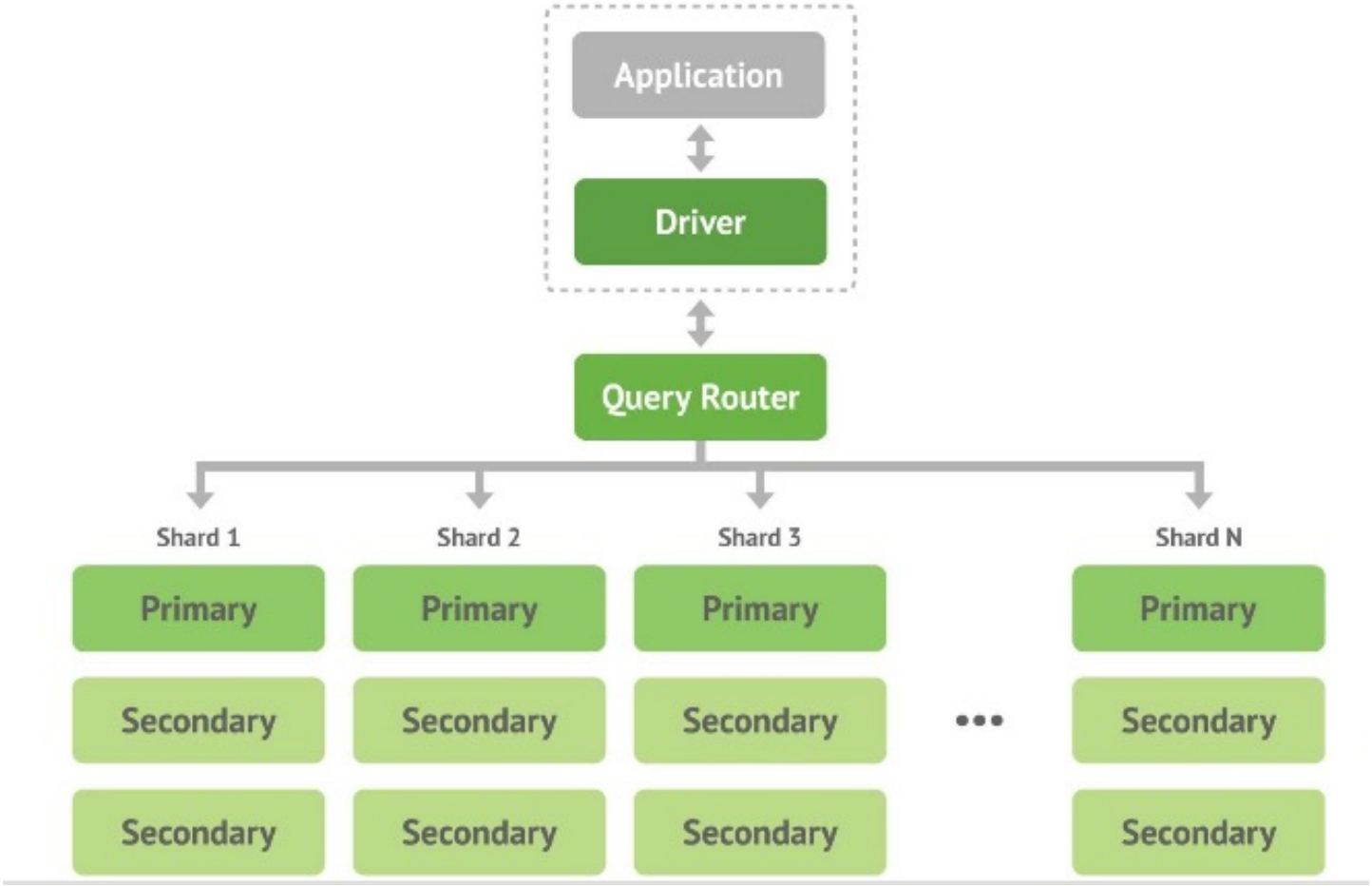
ÍNDICES

- Consideraciones:
  - Los índices se actualizan tras cada *Insert/Update/Delete* (se añade sobrecarga).
  - Sólo crear índices cuyo beneficio en consultas supere al perjuicio en escrituras (gran número de *Reads*, bajo número de actualizaciones).
- Otros índices:
  - Índices Hash
  - Índices para geoposicionamiento
- Más sobre índices en https://docs.mongodb.com/manual/indexes/

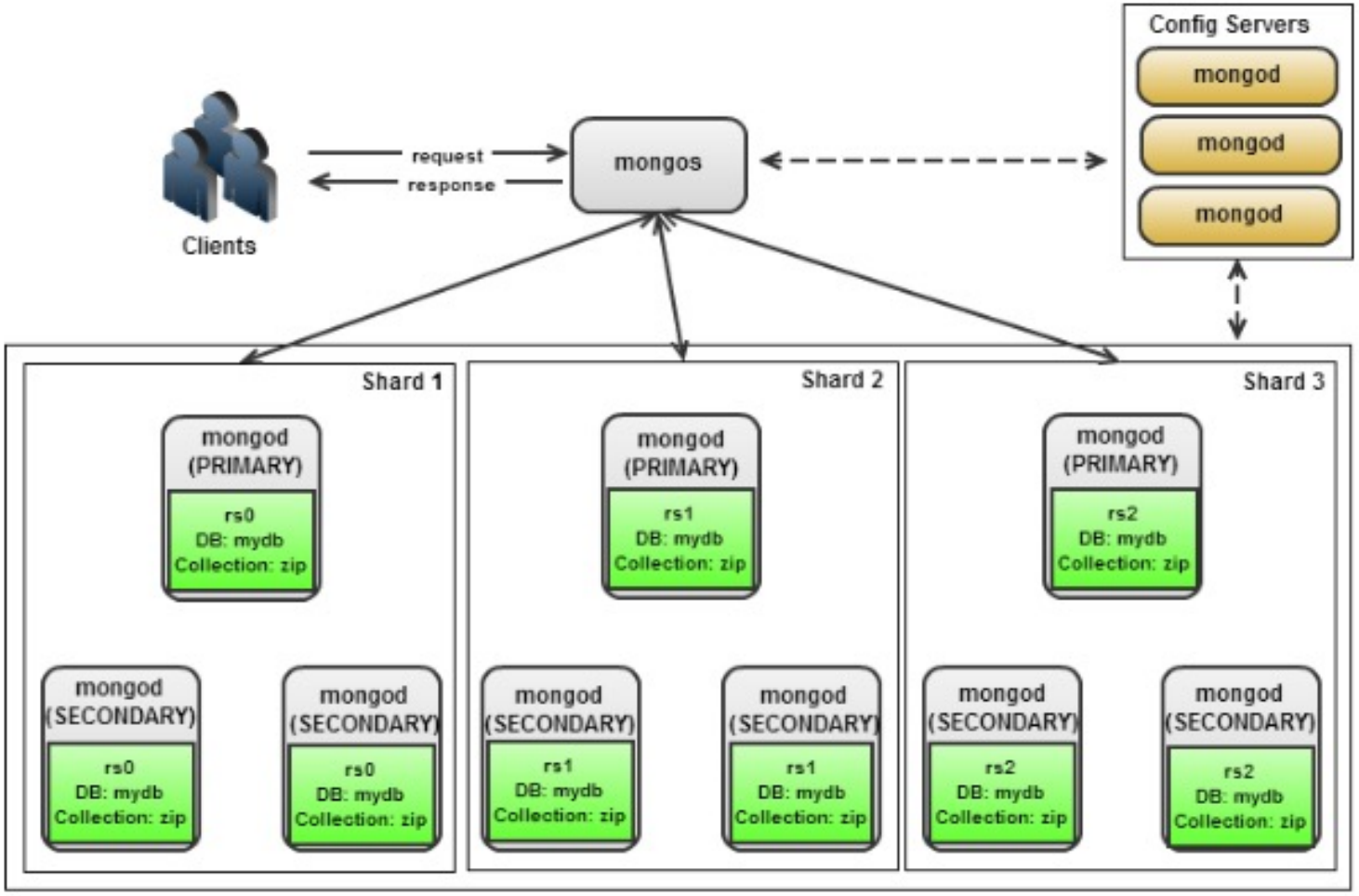
ARQUITECTURA DE CLUSTERING

- Los datos en MongoDB se almacenan en *clusters*
  - Datos repartidos entre los nodos: *sharding*
  - Escalado horizontal
- En cada *shard* se realiza replicación de los datos:
  - *Replica set*: conjunto de réplica para un *shard* en un *cluster*
  - Un elemento primario (*primary*) y varios secundarios (*secondary*)

ARQUITECTURA DE CLUSTERING



ARQUITECTURA DE CLUSTERING



ARQUITECTURA DE CLUSTERING

- Conceptos básicos:
  - *mongod*: Proceso primario (demonio) instanciable para el acceso a los datos.
  - *mongos*: Servicio de enrutado entre la aplicación y la base de datos.
  - *config.server*: Servidor de metadatos.
  - *Replica set*: Grupo de procesos de *mongod* que mantienen el mismo dataset, proporcionando redundancia y alta disponibilidad. Son la base de todos los despliegues de producción.
    - *Primary*: Copias principales.
    - *Secondary*: Copias secundarias de *primary*.
    - *Arbiter*: Si *primary* cae, decide qué *secondary* le sustituye. No mantiene copias de los datos.
  - *Shard*: Fragmento normalmente asociado a un *cluster* que almacena una parte de los datos. Contiene un *replica set*.

ARQUITECTURA DE CLUSTERING

- Conceptos básicos:
  - Los clientes se conectan a un *router* (servicio *mongos*) de la base de datos.
  - El servicio *mongos* se comunica con los *config.server* para determinar dónde están los datos requeridos, o dónde escribir los nuevos.
  - A través de *mongos*, el cliente se conecta con el *shard* específico. Cada *shard* tendrá un proceso *mongod* primario y de cero a varios procesos *mongod* secundarios.

ARQUITECTURA DE CLUSTERING

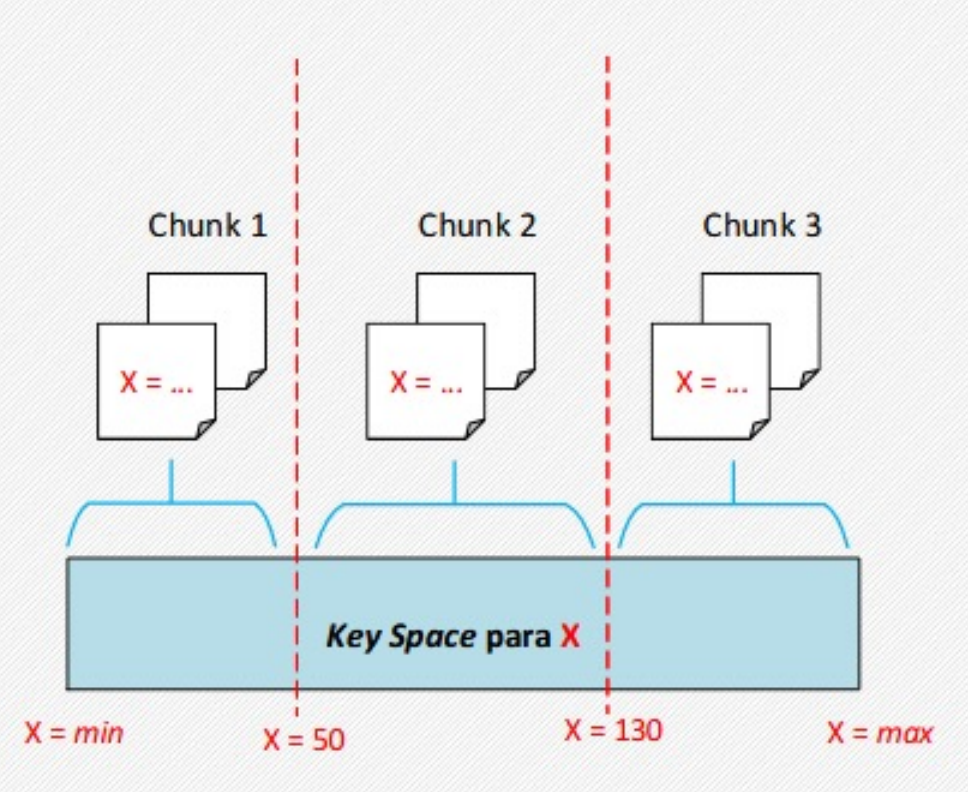
- *Sharding*:
  - Método para repartir los datos entre diferentes máquinas.
  - Cada documento está en una partición (*shard*):
    - La partición no es una sola máquina, sino un *replica set*.
  - Distribución de datos definida por la *shard key*:
    - Uno o varios campos que existan en todos los documentos, **han de estar indexados.**
    - Definida previamente y sin posibilidad de cambiarla.
    - Elección importante (define la distribución de información).
    - Los "trozos" en que se dividen los documentos de una colección se denominan *chunks*.

ARQUITECTURA DE CLUSTERING

- *Shard keys* y *chunks*:
  - *Range based sharding*: cada *chunk* tiene asociada una *shard key* mínima y una máxima.
    - Documentos con *shard key* similar probablemente estarán en el mismo *shard*.
    - Si un rango de claves contiene muchos documentos habrá una sobrecarga en un *shard*.
  - *Hash based sharding*: no existen rangos, cada documento se reparte de forma aleatoria.
    - Se consigue un buen equilibrio entre los *shards*.
    - Las consultas por rangos sobre la *shard key* es más lenta (accede a varios *shards*).

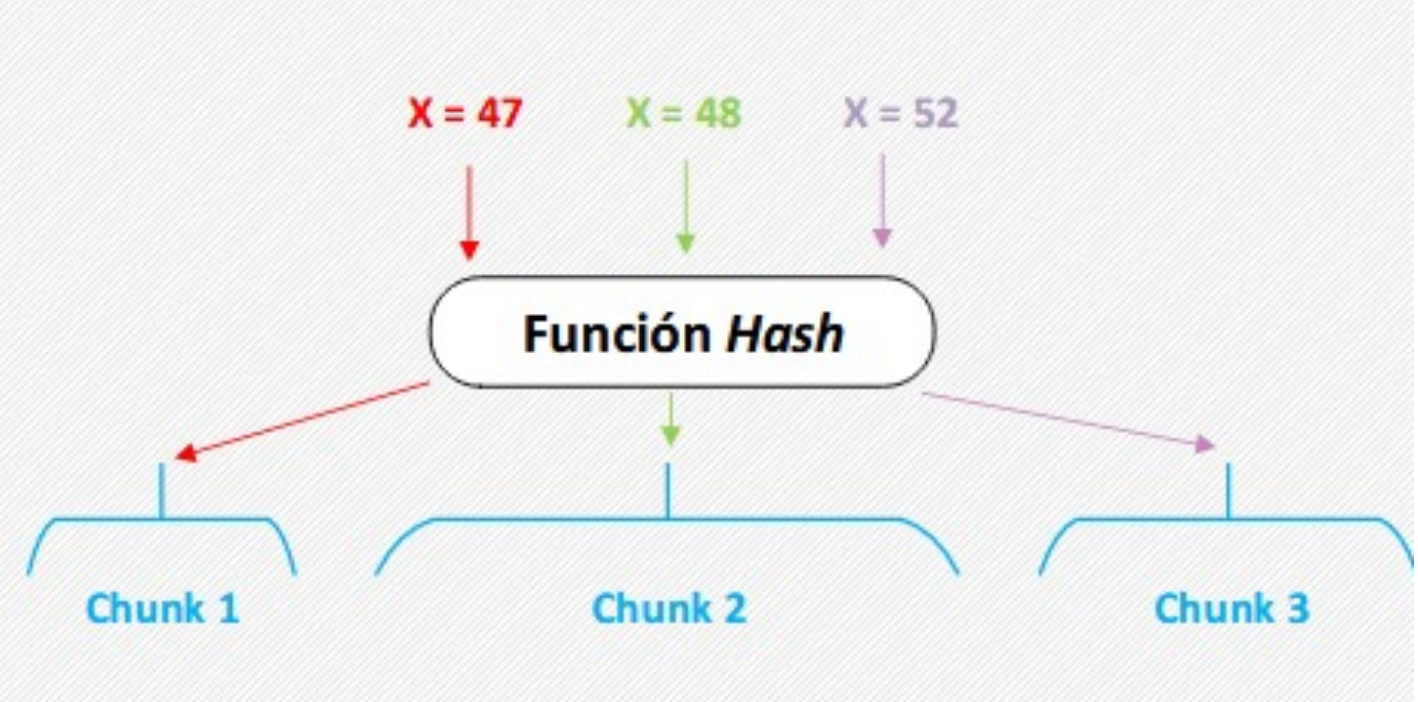
ARQUITECTURA DE CLUSTERING

- *Range based sharding:*



ARQUITECTURA DE CLUSTERING

- *Hash based sharding:*



ARQUITECTURA DE CLUSTERING

- *Shard key*. Implicaciones:
  - Si en una consulta se incluye la *shard key* sólo se consultarán los *shards* que contengan los documentos requeridos.
  - Si en una consulta no se incluye la *shard key* se realizará una búsqueda secuencial en todos los *shards* (mucho menos eficiente).

ARQUITECTURA DE CLUSTERING

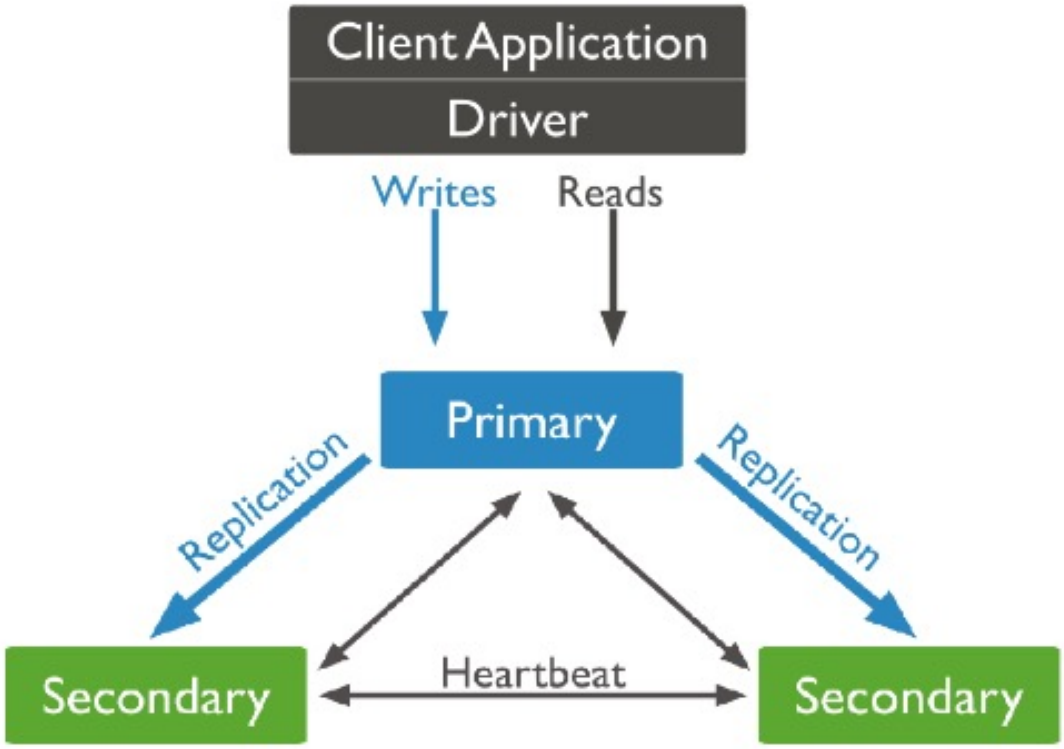
- *Sharding*. Servidores de configuración:
  - Almacenan metadatos relativos a los *chunks* que se almacenan en cada *shard*, así como la distribución de las *shard keys*.
  - Cada *sharded cluster* necesita su fichero de configuración propio.
  - Se leen cuando se inicia un nuevo servicio *mongos* o cuando cambian los metadatos.
  - Los metadatos se almacenan en la base de datos *config*.

ARQUITECTURA DE CLUSTERING

- *Sharding*. Requisitos mínimos:
  - Un *replica set* con 3 miembros como mínimo.
  - 3 servidores de configuración como *replica set*.
  - Un servicio *mongos* como mínimo.
    - Enruta *queries* de lectura y escritura al *shard* correspondiente.
    - Establece un cursor a los resultados de cada *query*.
    - Los metadatos sobre los *chunks* se guardan en caché (acceso más rápido).

ARQUITECTURA DE CLUSTERING

- Réplicas:



ARQUITECTURA DE CLUSTERING

- Réplicas:
  - Cada réplica es controlada por un servicio *mongod*.
  - Operaciones de escritura únicamente sobre el primario.
    - Las de lectura por defecto también, aunque se pueden configurar para realizarse sobre los secundarios.
  - Los cambios se propagan del primario a los secundarios.
  - *Heartbeat*: comunicación (cada 2 segundos) entre nodos del mismo *replica set* para comprobar que siguen activos.

ARQUITECTURA DE CLUSTERING

- Réplicas:
  - Si un secundario cae, no ocurre nada.
  - Si un primario cae (10 segundos sin responder):
    - Los secundarios eligen un nuevo primario
    - A través de votos
    - Si hay empate, el nodo *arbiter* decide (su función es únicamente de votación, no contiene datos y no puede cambiar de rol)
    - Para definir un *arbiter*: "*arbiterOnly:true*"
  - Si el primario se levanta posteriormente, pasa a ser secundario

ARQUITECTURA DE CLUSTERING

- Réplicas (otras características):
  - No proporciona balanceo de carga (todos los elementos del *replica set* contienen todos los datos de ese *shard*)
  - Redundancia y alta disponibilidad
  - Máximo de 50 miembros en un *replica set*
  - Máximo de 7 miembros con voto en un *replica set* (ha de ser un número impar, o si es par se ha de definir un *arbiter*)

ARQUITECTURA DE CLUSTERING

- Réplicas. Elección de primario:
  - Se elige como primario al que tenga los datos más adelantados.
  - Si todos están al día: datos de configuración (*priority*):
    - Priority:1 (opción por defecto, neutro)
    - Priority:0 (nunca será elegido)
    - Valores de probabilidad de elección entre 0 y 1

ARQUITECTURA DE CLUSTERING

- Réplicas. Casos especiales de secundarios:
  - Priority 0:
    - No pueden ser elegidos como primarios
    - Almacenan datos y votan igual que el resto
  - *Hidden replica set*:
    - No pueden ser elegidos como primarios (prioridad 0)
    - Almacenan datos y votan igual que el resto
    - No pueden ser leídos por la aplicación cliente
    - Útiles para *backups*
  - *Delayed replica set*:
    - Almacenan los datos con cierto retraso predefinido, votan igual que el resto
    - Similar a los *hidden replica sets* (no accesible al cliente, útiles para *backup* y *snapshots*)

ARQUITECTURA DE CLUSTERING

- Réplicas. Lectura de datos:
  - Opciones especificadas en la propia operación:

```
db.<collection>.find(...).readPref(<option>)
```

    - "primary": Opción por defecto, sólo lee del primario.
    - "primaryPreferred": Intenta leer del primario, pero si no es posible lo hace de un secundario.
    - "secondary": Lee sólo del secundario.
    - "secondaryPreferred": Usa los secundarios, pero si no es posible utiliza el primario (casos que supongan mucha carga y con poca necesidad de consistencia).
    - "nearest": Se utiliza el más cercano por localización (para casos con servidores repartidos en lugares muy alejados).

ARQUITECTURA DE CLUSTERING

- Réplicas. Estrategia de almacenamiento:
  - Se realizan *snapshots* en memoria de los últimos datos escritos o modificados.
  - *Checkpoints*: cada 60 segundos o al alcanzar los 2GB se escribe el *snapshot* en disco (nueva copia perdurable).
    - El antiguo *checkpoint* es válido hasta que el nuevo se escriba entero.
    - Permite recuperar en caso de error.
  - *Journal*: Se escriben todas las transacciones del *snapshot* en un *log* para poder recuperar datos escritos entre dos *checkpoint*.

ARQUITECTURA DE CLUSTERING

- Réplicas. Escritura de datos:
  - Distintos niveles de *WriteConcern*:
    - Sin confirmación: la aplicación cliente supone que se ha escrito el dato.
    - Confirmación de escritura únicamente en primario.
    - Confirmación de escritura en primario y uno o varios secundarios.
    - Confirmación de que se haya escrito o no en el *Journal*.

ARQUITECTURA DE CLUSTERING

- Más información:
  - *Sharding*: https://docs.mongodb.com/manual/sharding/
  - Administración y configuración: https://docs.mongodb.com/manual/administration/configuration/
  - Monitorizacion: https://docs.mongodb.com/manual/administration/monitoring

OTRAS CARACTERÍSTICAS

- Motores de almacenamiento:
  - MMAPv1:
    - Motor por defecto hasta la versión 3.2
    - Mejora de rendimiento con soporte de bloqueo a nivel de colección a partir de la versión 3.0
    - Eliminado en la versión 4.2
  - Wired Tiger:
    - Motor por defecto a partir de la versión 3.2
    - Bloqueo a nivel de documento
      - Mejora de rendimiento en carga de escritura (entre 7 y 10 veces)
    - Compresión de datos (hasta el 80%): reducción de costes
    - Reducción de latencia en las consultas
  - In-Memory:
    - A partir de la versión 3.2.6
    - Latencias de datos más predecibles
    - Requiere mucho espacio en memoria

OTRAS CARACTERÍSTICAS

- Monitorización de consultas:
  - Utilización: `db.<collection>.<method>(...).explain(<detail>)`
  - Sobre los métodos "aggregate", "count", "distinct", "find", "findAndModify", "delete" y "update".
  - Según el nivel de detalle:
    - queryPlanner (por defecto).
    - executionstats: información mucho más detallada (número de documentos, tiempo requerido…).
    - allPlansExecution: información sobre todos los posibles planes de ejecución (no sólo el ganador).

OTRAS CARACTERÍSTICAS

- Monitorización de consultas. Ejemplo:
  - Supongamos la base de datos "pruebas" y la colección "coord" que almacena coordenadas en forma de tuplas (x,y).
  - Insertamos documentos en la colección:
    - for (var i=0; i < 200000; i++) { db.coord.insert({x:i, y:i/2}) }
  - Se ejecuta la siguiente consulta monitorizada:
    - db.coord.find({x:15.23}).explain()

OTRAS CARACTERÍSTICAS

- Monitorización de consultas. Ejemplo:
  - Al no indicar nivel de detalle, devuelve "queryPlanner":

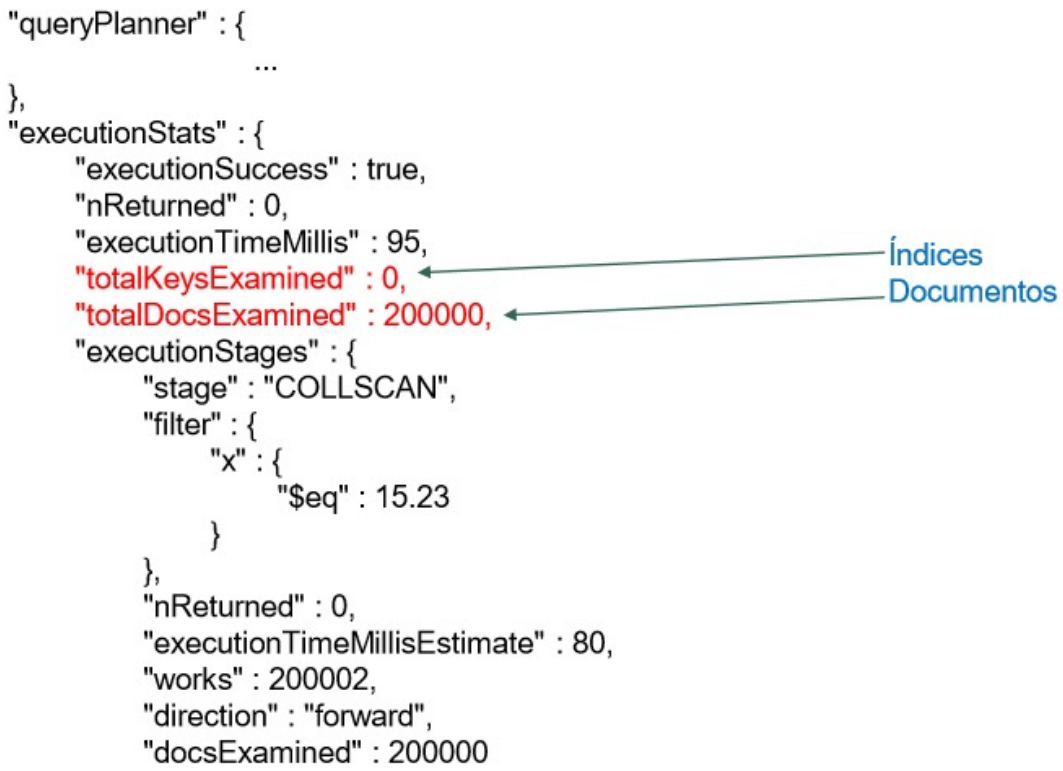

```
{
        "queryPlanner" : {
                "plannerVersion" : 1,
                "namespace" : "pruebas.coord",
                "indexFilterSet" : false,
                "parsedQuery" : {       "x" : {"$eq" : 15.23 }    },
                "winningPlan" : {
                        "stage" : "COLLSCAN",
                        "filter" : {"x" : {  "$eq" : 15.23 } },
                        "direction" : "forward"
                },
                "rejectedPlans" : [ ]
        },
        "serverInfo" : {
                "host" : "DESKTOP-2250LEB",
                "port" : 27017,
                "version" : "3.4.10",
                "gitVersion" : "078f28920cb24de0dd479b5ea6c66c644f6326e9"
        },
        "ok" : 1
}
```

OTRAS CARACTERÍSTICAS

- Monitorización de consultas. Ejemplo:
  - QueryPlanner:
    - General: *query* en formato interno (*parsedQuery*), nombre de la base de datos y colección.
    - winningPlan: plan escogido (cómo se organiza la búsqueda de documentos).
      - Stage:COLLSCAN indica que se ha recorrido la colección entera, sin índices
    - rejectedPlans: otros posibles planes que han sido desechados por menor eficiencia.
    - serverInfo: lugar donde se ha desarrollado la consulta, y datos sobre la versión de Mongo.

OTRAS CARACTERÍSTICAS

- Monitorización de consultas. Ejemplo:
  - ExecutionStats: 0 documentos devueltos, 0 índices, 200000 documentos examinados



OTRAS CARACTERÍSTICAS

- Más sobre monitorización de consultas:
  - https://docs.mongodb.com/manual/tutorial/analyze-query-plan/
  - https://docs.mongodb.com/manual/reference/command/explain/

OTRAS CARACTERÍSTICAS

- Clientes gráficos:
  - Robomongo: http://robomongo.org
  - Mongo Express: https://github.com/mongo-express/mongo-express
  - NoSQL Booster: https://nosqlbooster.com/

OTRAS CARACTERÍSTICAS

- Cliente Python:
  - PyMongo:
    - https://api.mongodb.com/python/current/tutorial.html
    - https://www.mongodb.com/blog/post/getting-started-with-python-and-mongodb

CONCLUSIONES

- Ventajas de MongoDB:
  - Gran flexibilidad en los esquemas.
  - Lenguaje de consulta y manipulación (CRUD) sencillo.
  - Facilidad de integración gracias al uso de BSON (fácilmente traducible a JSON).
  - Accesibilidad a los datos:
    - Posibilidad de realizar lecturas en instancias secundarias.

CONCLUSIONES

- Desventajas de MongoDB:
  - Aplicaciones clientes más complejas (esquemas flexibles, desnormalizados y dinámicos).
  - No garantiza ACID:
    - Consistencia eventual (se pueden hacer lecturas de nodos secundarios aún no actualizados).
    - Sin soporte transaccional.

LECTURAS PARA PREPARAR EL TEMA

- Libro *MongoDB: The Definitive Guide, 3rd Edition*:
  - https://learning.oreilly.com/library/view/mongodb-the-definitive/9781491954454/
- Documentación oficial MongoDB:
  - https://docs.mongodb.com/
- Los enlaces de los libros son accesibles para usuarios autenticados en UNED